

```
"""
```

```
=====
```

```
=====
```

```
Problema 02 – Análise de Estabilidade e Custo em Sistemas de Abastecimento
```

```
=====
```

```
=====
```

```
Disciplina : Algoritmos e Programação
```

```
Data      : 12/05/2026
```

```
=====
```

```
=====
```

```
"""
```

```
import random
```

```
import sys
```

```
#
```

```
_____
```

```
_____
```

```
# Tentativa de importar matplotlib (necessário para os gráficos)
```

```
#
```

```
_____
```

```
_____
```

```
try:
```

```
    import matplotlib.pyplot as plt
```

```
    import matplotlib.patches as mpatches
```

```
    from matplotlib.gridspec import GridSpec
```

```
    MATPLOTLIB_OK = True
```

```
except ImportError:
```

```
    MATPLOTLIB_OK = False
```

```
    print("[AVISO] matplotlib não encontrado. Instale com: pip install matplotlib")
```

```
    print("      Os gráficos não serão gerados.\n")
```

```
#
```

```
=====
```

```
=====
```

```
# FUNÇÕES AUXILIARES DE ENTRADA
```

```
#
```

```
=====
```

```
=====
```

```
def ler_inteiro(mensagem: str, minimo: int = 1) -> int:
```

```
    """Lê um inteiro ≥ mínimo, rejeita entradas inválidas."""
```

```

while True:
    try:
        valor = int(input(mensagem))
        if valor < minimo:
            print(f" X O valor deve ser pelo menos {minimo}. Tente novamente.")
        else:
            return valor
    except ValueError:
        print(" X Entrada inválida. Digite um número inteiro.")

```

```

def ler_float(mensagem: str, minimo: float = 0.0, exclusivo: bool = False) -> float:
    """Lê um float ≥ mínimo (ou > mínimo se exclusivo=True), rejeita inválidos."""
    while True:
        try:
            valor = float(input(mensagem))
            if exclusivo and valor <= minimo:
                print(f" X O valor deve ser maior que {minimo}. Tente novamente.")
            elif not exclusivo and valor < minimo:
                print(f" X O valor deve ser ≥ {minimo}. Tente novamente.")
            else:
                return valor
        except ValueError:
            print(" X Entrada inválida. Digite um número.")

```

```

def ler_percentual(mensagem: str) -> float:
    """Lê um percentual entre 0 e 100 (inclusive)."""
    while True:
        valor = ler_float(mensagem, minimo=0.0)
        if valor > 100.0:
            print(" X Percentual deve ser entre 0 e 100.")
        else:
            return valor

```

#

FUNÇÕES DE CÁLCULO

#

```
def calcular_volume_entregue(demanda: float, perda_pct: float) -> float:
    """Volume efetivamente entregue = demanda × (1 – perda%)."""
    return demanda * (1.0 - perda_pct / 100.0)
```

```
def calcular_volume_perdido(demanda: float, perda_pct: float) -> float:
    """Volume perdido na tubulação = demanda × perda%. """
    return demanda * (perda_pct / 100.0)
```

```
def calcular_custo_diario(demanda: float, custo_por_m3: float) -> float:
    """Custo diário de bombeamento = demanda × custo/m³. """
    return demanda * custo_por_m3
```

```
def calcular_autonomia(volume_inicial: float, demanda: float) -> float:
    """
    Autonomia em dias = volume_inicial / demanda.
    Retorna infinito se demanda for zero (reservatório nunca esgota).
    """
    if demanda <= 0:
        return float("inf")
    return volume_inicial / demanda
```

```
def classificar_risco(autonomia: float) -> str:
    """Classifica o risco de desabastecimento com base na autonomia. """
    if autonomia > 3:
        return "Baixo"
    elif autonomia >= 1:
        return "Médio"
    else:
        return "Alto"
```

```
#
```

```
# COLETA DE DADOS
```

```
#
```

```

def coletar_dados() -> list[dict]:
    """Solicita ao usuário os dados de cada bairro e retorna lista de dicts."""
    n = ler_inteiro("\nQuantos bairros serão analisados? ", minimo=1)
    bairros = []

    for i in range(1, n + 1):
        print(f"\n{'-'*50}")
        print(f" Bairro {i} de {n}")
        print(f"{'-'*50}")

        nome = input(" Nome do bairro: ").strip()
        while not nome:
            print(" X O nome não pode ser vazio.")
            nome = input(" Nome do bairro: ").strip()

        demanda = ler_float(" Demanda diária estimada (m³): ", minimo=0.0,
exclusivo=True)
        capacidade = ler_float(" Capacidade do reservatório (m³): ", minimo=0.0,
exclusivo=True)
        volume_inicial = ler_float(" Volume inicial disponível (m³): ", minimo=0.0)
        perda_pct = ler_percentual(" Percentual de perda na tubulação (%): ")
        custo_por_m3 = ler_float(" Custo de bombeamento por m³ (R$): ", minimo=0.0)

        # Volume inicial não pode exceder a capacidade
        if volume_inicial > capacidade:
            print(f" ⚠ Volume inicial ({volume_inicial:.2f} m³) excede a capacidade "
                f"({capacidade:.2f} m³). Será ajustado para a capacidade.")
            volume_inicial = capacidade

        bairros.append({
            "nome": nome,
            "demanda": demanda,
            "capacidade": capacidade,
            "vol_inicial": volume_inicial,
            "perda_pct": perda_pct,
            "custo_por_m3": custo_por_m3,
        })

    return bairros

```

#

```
# PROCESSAMENTO
```

```
#
```

```
def processar_bairros(bairros: list[dict]) -> list[dict]:
    """Calcula todos os indicadores para cada bairro."""
    resultados = []
    for b in bairros:
        vol_entregue = calcular_volume_entregue(b["demanda"], b["perda_pct"])
        vol_perdido = calcular_volume_perdido(b["demanda"], b["perda_pct"])
        custo_diario = calcular_custo_diario(b["demanda"], b["custo_por_m3"])
        autonomia = calcular_autonomia(b["vol_inicial"], b["demanda"])
        risco = classificar_risco(autonomia)

        resultados.append({
            **b,
            "vol_entregue": vol_entregue,
            "vol_perdido": vol_perdido,
            "custo_diario": custo_diario,
            "autonomia": autonomia,
            "risco": risco,
        })
    return resultados
```

```
def identificar_mais_critico(resultados: list[dict]) -> dict:
    """Retorna o bairro com menor autonomia (mais crítico)."""
    return min(resultados, key=lambda r: r["autonomia"])
```

```
#
```

```
# SAÍDA – TABELA NO TERMINAL
```

```
#
```

```
def imprimir_tabela(resultados: list[dict]) -> None:
    """Imprime tabela formatada com os resultados por bairro."""
```

```
sep = "=" * 108
sep2 = "-" * 108
```

```
cabecalho = (
    f'{"Bairro":<18} | {"Demanda":>10} | {"Entregue":>10} | '
    f'{"Perdido":>10} | {"Custo/dia":>12} | {"Autonomia":>10} | {"Risco":<8}'
)
unidades = (
    f'{"":18} | {"(m³)":>10} | {"(m³)":>10} | '
    f'{"(m³)":>10} | {"(R$)":>12} | {"(dias)":>10} | {"":8}'
)
```

```
print(f"\n{sep}")
print(f"{'RESULTADOS POR BAIRRO':^108}")
print(sep)
print(cabecalho)
print(unidades)
print(sep2)
```

```
CORES_RISCO = {"Baixo": "✓", "Médio": "△", "Alto": "✖"}
```

```
for r in resultados:
    auto_str = f"{r['autonomia']:.2f}" if r["autonomia"] != float("inf") else "∞"
    icone = CORES_RISCO.get(r["risco"], "")
    print(
        f"{r['nome']:<18} | {r['demanda']:>10.2f} | {r['vol_entregue']:>10.2f} | '
        f'{r['vol_perdido']:>10.2f} | {r['custo_diario']:>12.2f} | '
        f'{auto_str:>10} | {icone} {r['risco']:<6}'
    )
```

```
print(sep)
```

```
critico = identificar_mais_critico(resultados)
print(f"\n🔴 Bairro mais crítico: {critico['nome']} "
      f'(autonomia: {critico['autonomia']:.2f} dias | risco: {critico['risco']})")
print()
```

```
#
```

```
# SAÍDA – GRÁFICOS
```

```
#
```

```
COR_RISCO = {"Baixo": "#2ecc71", "Médio": "#f39c12", "Alto": "#e74c3c"}
```

```
def gerar_graficos_estaticos(resultados: list[dict]) -> None:
```

```
    """Gera os 3 gráficos obrigatórios da análise estática."""
```

```
    nomes = [r["nome"] for r in resultados]
```

```
    n = len(nomes)
```

```
    x = list(range(n))
```

```
    largura = 0.35
```

```
    fig = plt.figure(figsize=(18, 14))
```

```
    fig.suptitle("Sistema de Abastecimento de Água – Análise Estática",  
                fontsize=16, fontweight="bold", y=0.98)
```

```
    gs = GridSpec(2, 2, figure=fig, hspace=0.45, wspace=0.35)
```

```
    # — Gráfico 1: Demanda vs Volume Entregue
```

```
    ax1 = fig.add_subplot(gs[0, :])      # ocupa toda a primeira linha
```

```
    bars_dem = ax1.bar([xi - largura/2 for xi in x], [r["demanda"] for r in resultados],  
                      largura, label="Demanda", color="#3498db", alpha=0.85)
```

```
    bars_ent = ax1.bar([xi + largura/2 for xi in x], [r["vol_entregue"] for r in resultados],  
                      largura, label="Vol. Entregue", color="#1abc9c", alpha=0.85)
```

```
    ax1.set_title("Demanda Diária vs Volume Efetivamente Entregue (m³)", fontsize=13)
```

```
    ax1.set_ylabel("Volume (m³)")
```

```
    ax1.set_xticks(x)
```

```
    ax1.set_xticklabels(nomes, rotation=30, ha="right")
```

```
    ax1.legend()
```

```
    ax1.grid(axis="y", linestyle="--", alpha=0.4)
```

```
    # Rótulos sobre as barras
```

```
    for bar in bars_dem:
```

```
        ax1.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,  
                f"{bar.get_height():.1f}", ha="center", va="bottom", fontsize=8)
```

```
    for bar in bars_ent:
```

```
        ax1.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,  
                f"{bar.get_height():.1f}", ha="center", va="bottom", fontsize=8)
```

```
    # — Gráfico 2: Custo Diário por Bairro
```

```
    ax2 = fig.add_subplot(gs[1, 0])
```

```

cores_custo = [COR_RISCO[r["risco"]] for r in resultados]
bars_custo = ax2.bar(nomes, [r["custo_diario"] for r in resultados],
                    color=cores_custo, alpha=0.85, edgecolor="white")
ax2.set_title("Custo Diário de Bombeamento (R$)", fontsize=12)
ax2.set_ylabel("Custo (R$)")
ax2.set_xticklabels(nomes, rotation=30, ha="right")
ax2.grid(axis="y", linestyle="--", alpha=0.4)

for bar in bars_custo:
    ax2.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
            f"R$ {bar.get_height():.2f}", ha="center", va="bottom", fontsize=8)

# Legenda manual de risco no gráfico de custo
patches = [mpatches.Patch(color=v, label=k) for k, v in COR_RISCO.items()]
ax2.legend(handles=patches, title="Nível de Risco", fontsize=8)

```

```

# — Gráfico 3: Distribuição dos Níveis de Risco —————
ax3 = fig.add_subplot(gs[1, 1])
contagem_risco = {"Baixo": 0, "Médio": 0, "Alto": 0}
for r in resultados:
    contagem_risco[r["risco"]] += 1

labels = [k for k, v in contagem_risco.items() if v > 0]
valores = [v for v in contagem_risco.values() if v > 0]
cores = [COR_RISCO[k] for k in labels]

ax3.pie(valores, labels=labels, colors=cores, autopct="%1.1f%%",
        startangle=90, textprops={"fontsize": 10},
        wedgeprops={"edgecolor": "white", "linewidth": 1.5})
ax3.set_title("Distribuição dos Níveis de Risco", fontsize=12)

plt.savefig("graficos_estaticos.png", dpi=150, bbox_inches="tight")
plt.show()
print(" [✓] Gráficos estáticos salvos em 'graficos_estaticos.png'")

```

#

SIMULAÇÃO – 7 DIAS

#

```

def simular_7_dias(bairros: list[dict]) -> None:
    """
    Simula 7 dias consecutivos com demanda variando  $\pm 20\%$  aleatoriamente.
    Atualiza volumes dos reservatórios e indica dias críticos.
    Gera gráfico de linha com evolução dos volumes.
    """

    N_DIAS = 7
    VAR_FAIXA = 0.20 #  $\pm 20\%$ 

    print(f"\n{'='*70}")
    print(f"SIMULAÇÃO – 7 DIAS CONSECUTIVOS'^70}")
    print(f"{'='*70}")

    # Volumes iniciais e histórico
    volumes_atuais = [b["vol_inicial"] for b in bairros]
    historico = [[b["vol_inicial"]] for b in bairros] # histórico[bairro][dia]

    for dia in range(1, N_DIAS + 1):
        print(f"\n Dia {dia}:")
        criticos_hoje = []

        for i, b in enumerate(bairros):
            # Demanda variável  $\pm 20\%$ 
            fator = 1 + random.uniform(-VAR_FAIXA, VAR_FAIXA)
            demanda = b["demanda"] * fator

            # Volume entregue e novo volume do reservatório
            vol_entregue = calcular_volume_entregue(demanda, b["perda_pct"])
            volumes_atuais[i] -= vol_entregue # desconta do reservatório

            # Limita entre 0 e capacidade máxima
            volumes_atuais[i] = max(0.0, min(volumes_atuais[i], b["capacidade"]))

            autonomia = calcular_autonomia(volumes_atuais[i], demanda)
            risco = classificar_risco(autonomia)

            historico[i].append(volumes_atuais[i])

        status = "🔴 CRÍTICO" if risco == "Alto" else ("⚠️ Médio" if risco == "Médio" else "✅ OK")

        print(f"  {b['nome'][:18]} | dem={demanda:7.2f} m³ | "
              f"vol={volumes_atuais[i]:8.2f} m³ | aut={autonomia:2f} d | {status}")

```

```

    if risco == "Alto":
        criticos_hoje.append(b["nome"])

    if criticos_hoje:
        print(f"\n   △ Bairros em situação crítica no Dia {dia}: "
              f"{', '.join(criticos_hoje)}")
    else:
        print(f"\n   ✓ Nenhum bairro em situação crítica no Dia {dia}.")

```

— Gráfico de linha

```

if not MATPLOTLIB_OK:
    return

dias_x = list(range(0, N_DIAS + 1)) # 0 = situação inicial
fig, ax = plt.subplots(figsize=(12, 6))

cmap = plt.colormaps.get_cmap("tab10")
for i, b in enumerate(bairros):
    ax.plot(dias_x, historico[i], marker="o", linewidth=2,
            color=cmap(i % 10), label=b["nome"])
    # Anota o volume final
    ax.annotate(f"{historico[i][-1]:.1f}",
                xy=(N_DIAS, historico[i][-1]),
                xytext=(5, 0), textcoords="offset points",
                fontsize=8, color=cmap(i % 10))

ax.set_title("Evolução do Volume Disponível nos Reservatórios (7 dias)", fontsize=13)
ax.set_xlabel("Dia")
ax.set_ylabel("Volume (m³)")
ax.set_xticks(dias_x)
ax.set_xticklabels(["Inicial"] + [f"Dia {d}" for d in range(1, N_DIAS + 1)])
ax.legend(loc="upper right", fontsize=9)
ax.grid(linestyle="--", alpha=0.4)

plt.tight_layout()
plt.savefig("simulacao_7_dias.png", dpi=150, bbox_inches="tight")
plt.show()
print("\n [✓] Gráfico da simulação salvo em 'simulacao_7_dias.png'")

```

#

```
# PONTO DE ENTRADA
```

```
#
```

```
def main() -> None:
```

```
    print("┌" + "=" * 68 + "┐")
```

```
    print("│{: ^68}│".format("SISTEMA DE ANÁLISE DE ABASTECIMENTO DE ÁGUA"))
```

```
    print("│{: ^68}│".format("Algoritmos e Programação – Problema 02"))
```

```
    print("└" + "=" * 68 + "┘")
```

```
    # 1. Coleta de dados
```

```
    bairros = coletar_dados()
```

```
    # 2. Processamento
```

```
    resultados = processar_bairros(bairros)
```

```
    # 3. Tabela de resultados
```

```
    imprimir_tabela(resultados)
```

```
    # 4. Gráficos estáticos
```

```
    if MATPLOTLIB_OK:
```

```
        print("Gerando gráficos estáticos...")
```

```
        gerar_graficos_estaticos(resultados)
```

```
    else:
```

```
        print("[AVISO] Gráficos não disponíveis (matplotlib ausente).")
```

```
    # 5. Simulação de 7 dias
```

```
    resposta = input("\nDeseja executar a simulação de 7 dias? (s/n): ").strip().lower()
```

```
    if resposta in ("s", "sim", "y", "yes"):
```

```
        simular_7_dias(bairros)
```

```
    else:
```

```
        print("Simulação ignorada.")
```

```
    print("\n✓ Programa encerrado com sucesso.")
```

```
if __name__ == "__main__":
```

```
    main()
```